

Instituto Profesional
Virginio Gómez
Universidad de Concepción

Arquitectura de Software para un Campus Inteligente

Ingeniería de Ejecución en Computación e Informática
Semana 6: Lenguaje de Programación OO (0568)
Plataforma: C# y .NET 8
Proyecto Central: Sistema de Gestión de Lockers y Módulos IPVG

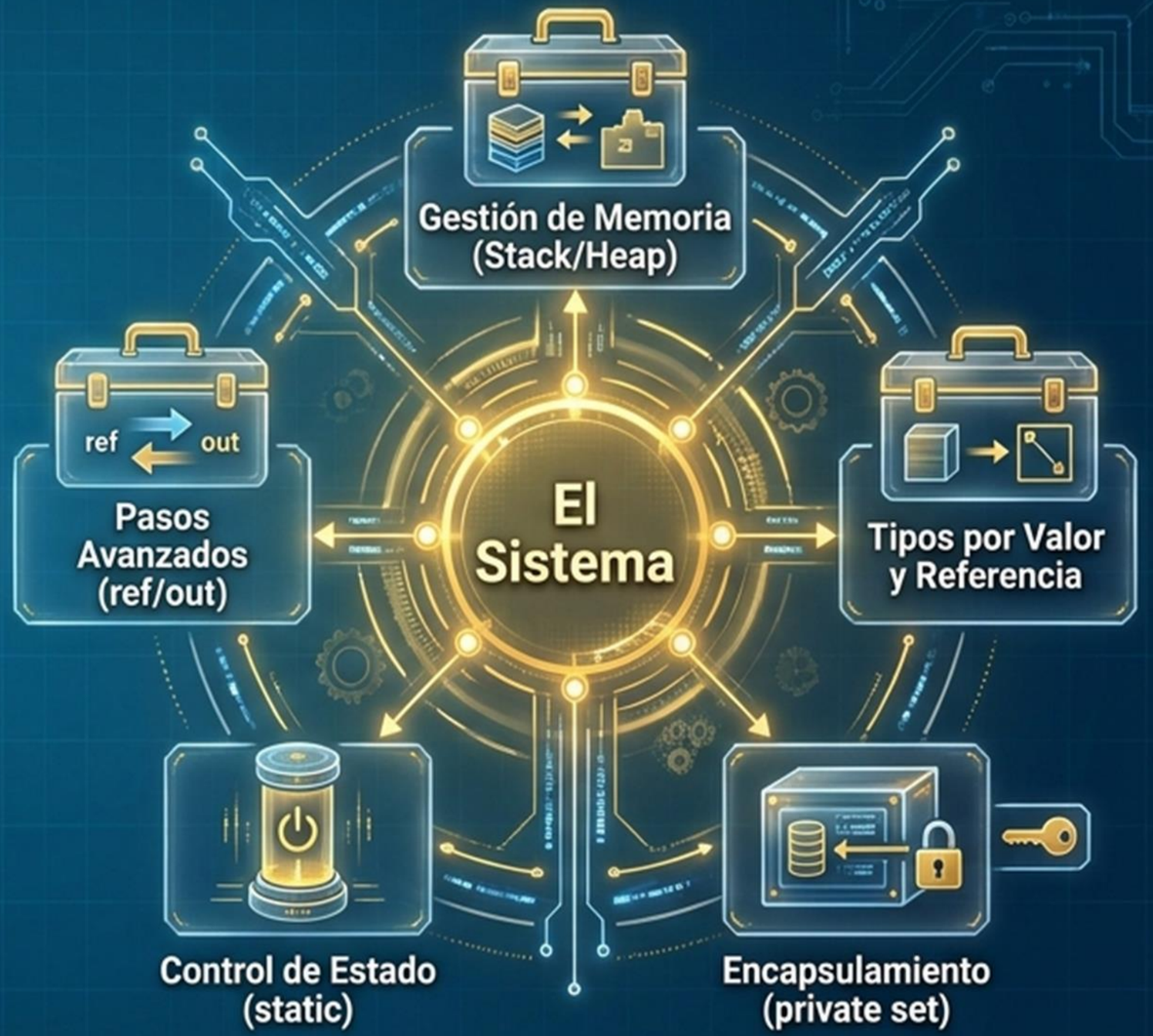
El Encargo Institucional: Modernizar la Infraestructura

El Encargo

La Sede IPVG requiere digitalizar la asignación y gestión de los lockers para los estudiantes.

El sistema debe ser rápido, seguro, trazable y, sobre todo, escalable para soportar la matrícula actual y futura.

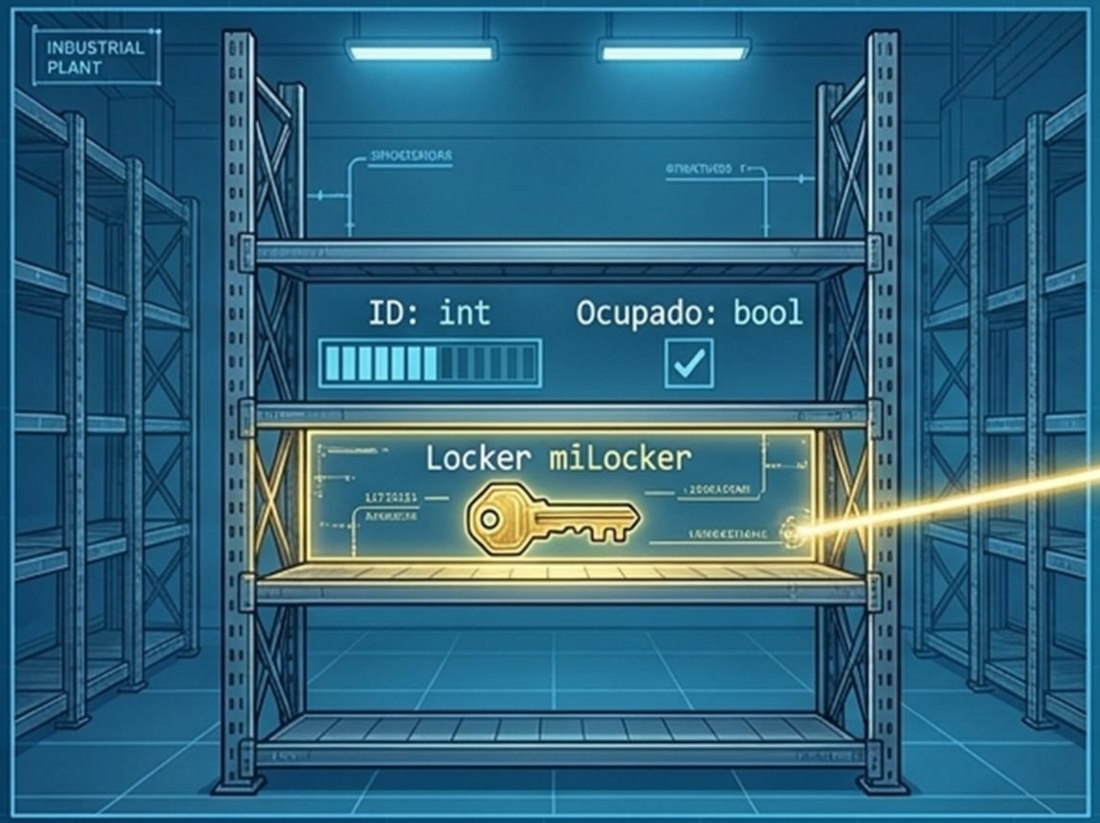
Inventario Técnico



Blueprints of a Smart Campus

Zonificación de Memoria: Stack vs. Heap

El Stack (Pila)



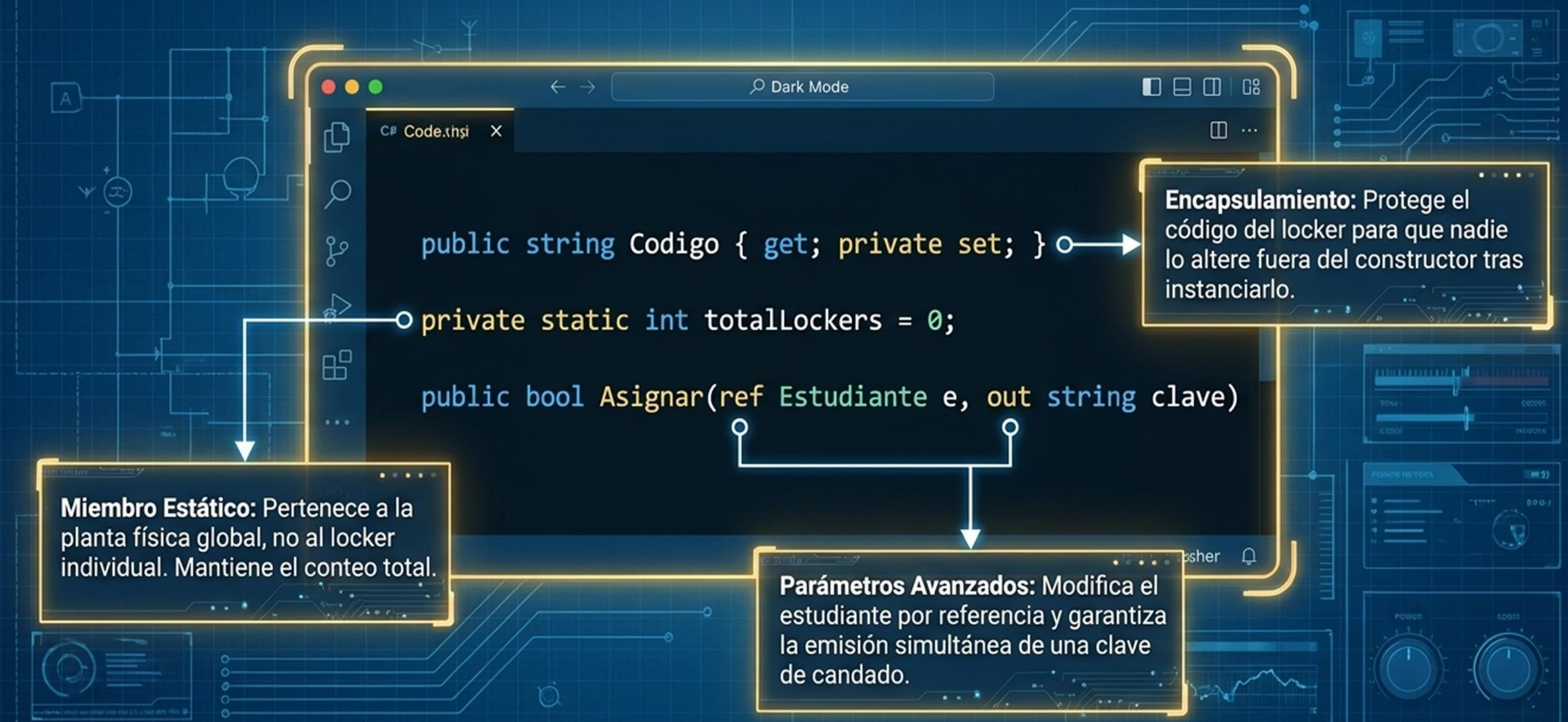
Rápido, estructurado, tamaño fijo.
Aquí viven los Tipos por Valor (structs, int, bool) y las Referencias (las llaves).

El Heap (Montículo)



Dinámico, administrado por el Garbage Collector.
Aquí viven los Tipos por Referencia.
Ej: new Locker("L-001").

Los Cimientos del Código: Clase Locker

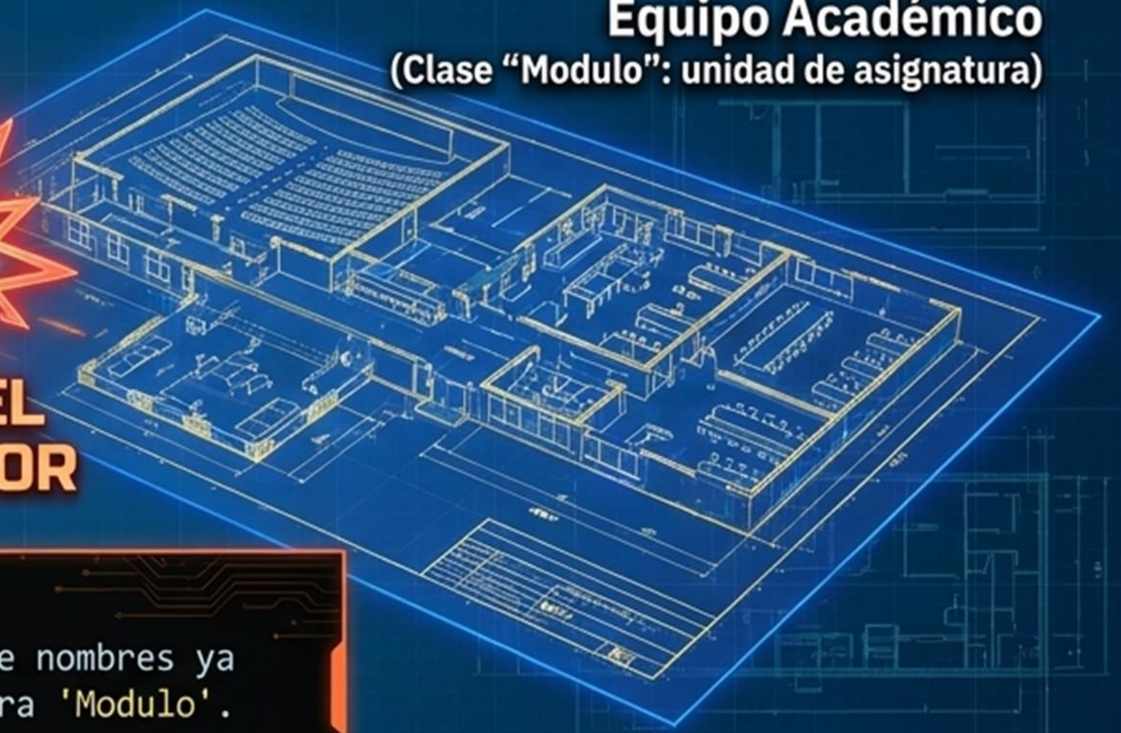


El Sistema Crece: Colisión Estructural

Equipo de Infraestructura
(Clase “Modulo”: bloque de edificio)



Equipo Académico
(Clase “Modulo”: unidad de asignatura)



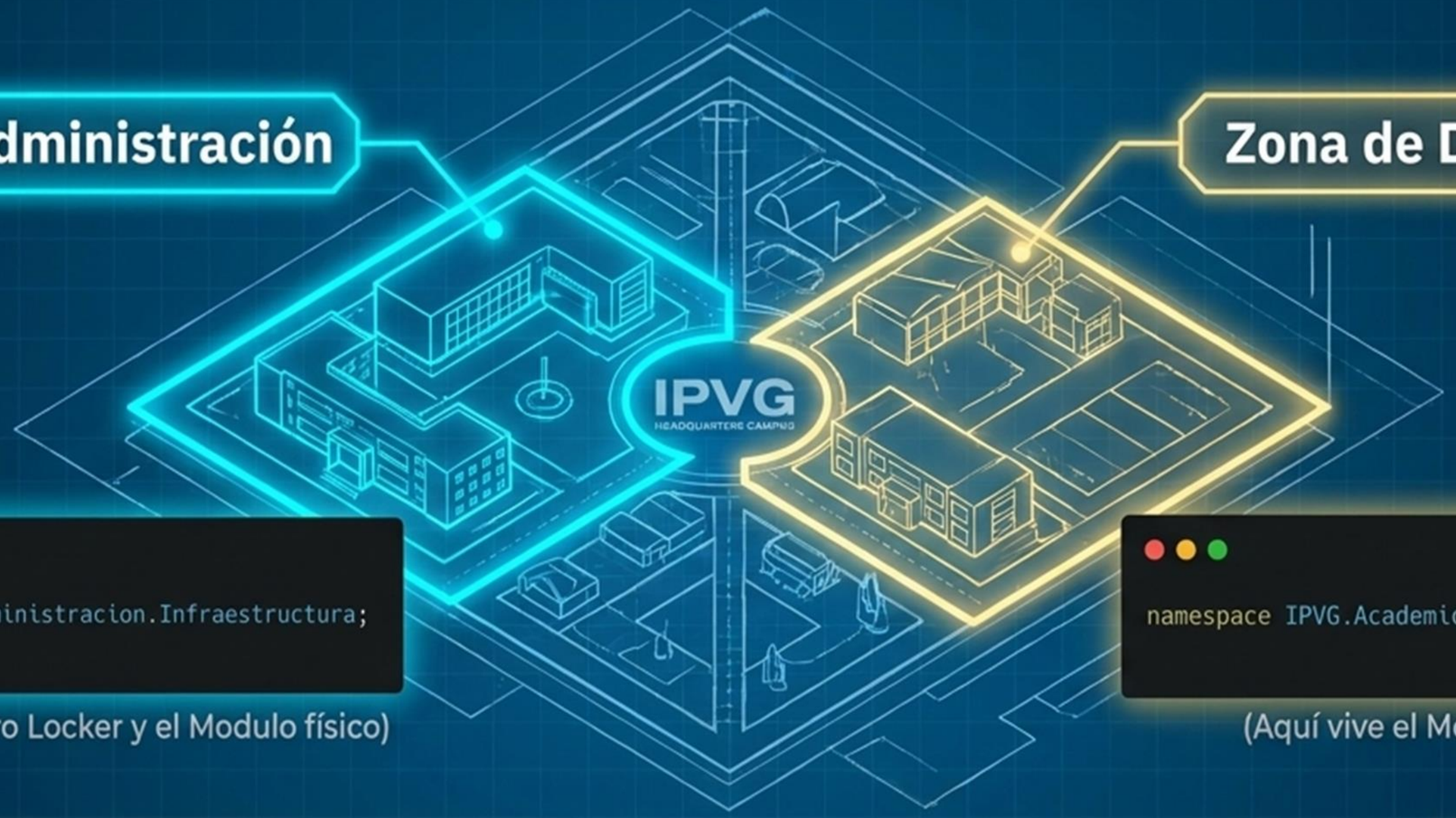
ERROR DEL COMPILADOR

```
Error CS0101: El espacio de nombres ya
contiene una definición para 'Modulo'.
```

¿Cómo evitamos reescribir todo el código y renombrar cientos de archivos ante esta colisión de nombres?

Solución Arquitectónica: Espacios de Nombres

Zona de Administración



Zona de Docencia

```
namespace IPVG.Administracion.Infraestructura;
```

(Aquí viven nuestro Locker y el Modulo físico)

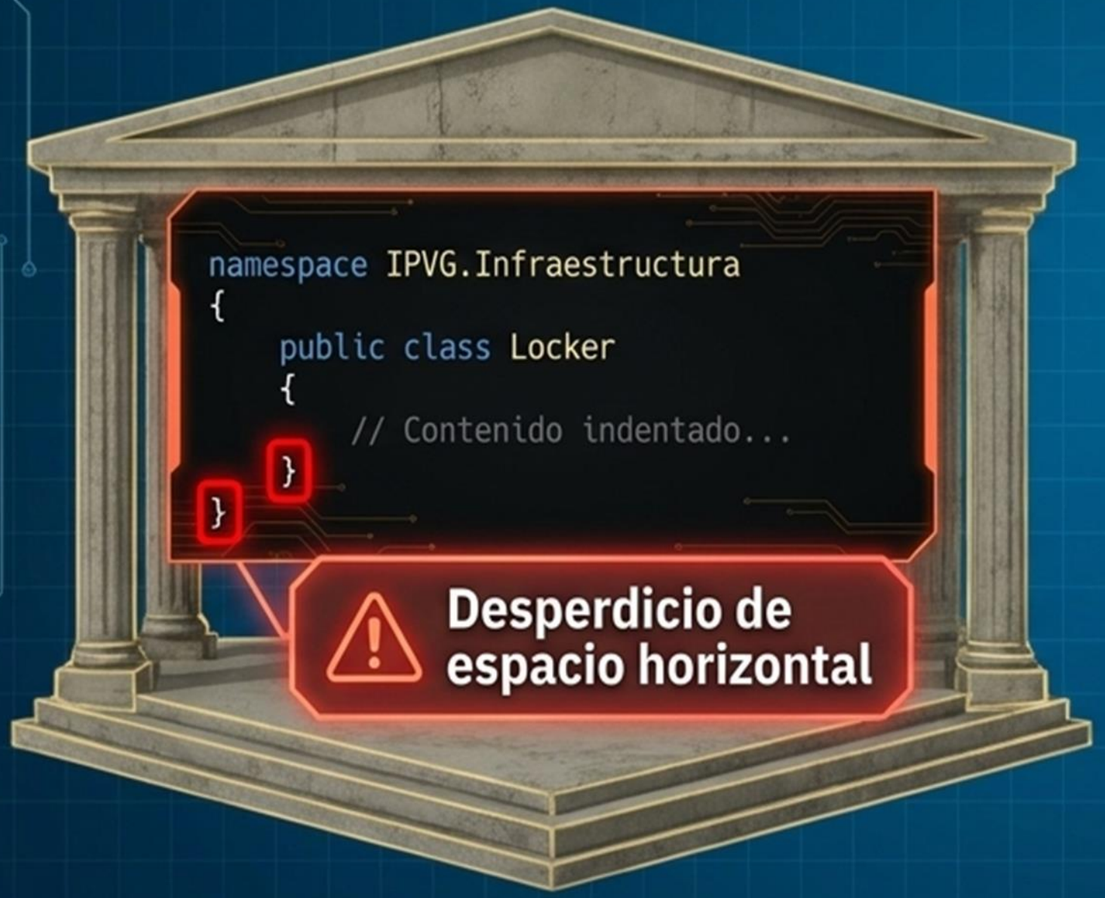
```
namespace IPVG.Academico.Asignaturas;
```

(Aquí vive el Modulo de estudio)

Concepto Clave: Los espacios de nombres organizan lógicamente las clases y previenen colisiones, permitiendo que dos clases se llamen exactamente igual siempre que residan en diferentes "pisos" del código.

Evolución Estructural: File-Scoped Namespaces

Block-Scoped (Tradicional)



File-Scoped (Moderno C# 10+)



Conectando las Zonas: Directivas y Alias

IPVG.Administracion.Infraestructura



Directiva Using

```
using IPVG.Administracion.Infraestructura;
```

Abre las puertas de esa zona

Program.cs
(Sistema Principal)

```
Lck miLocker = new Lck();
```



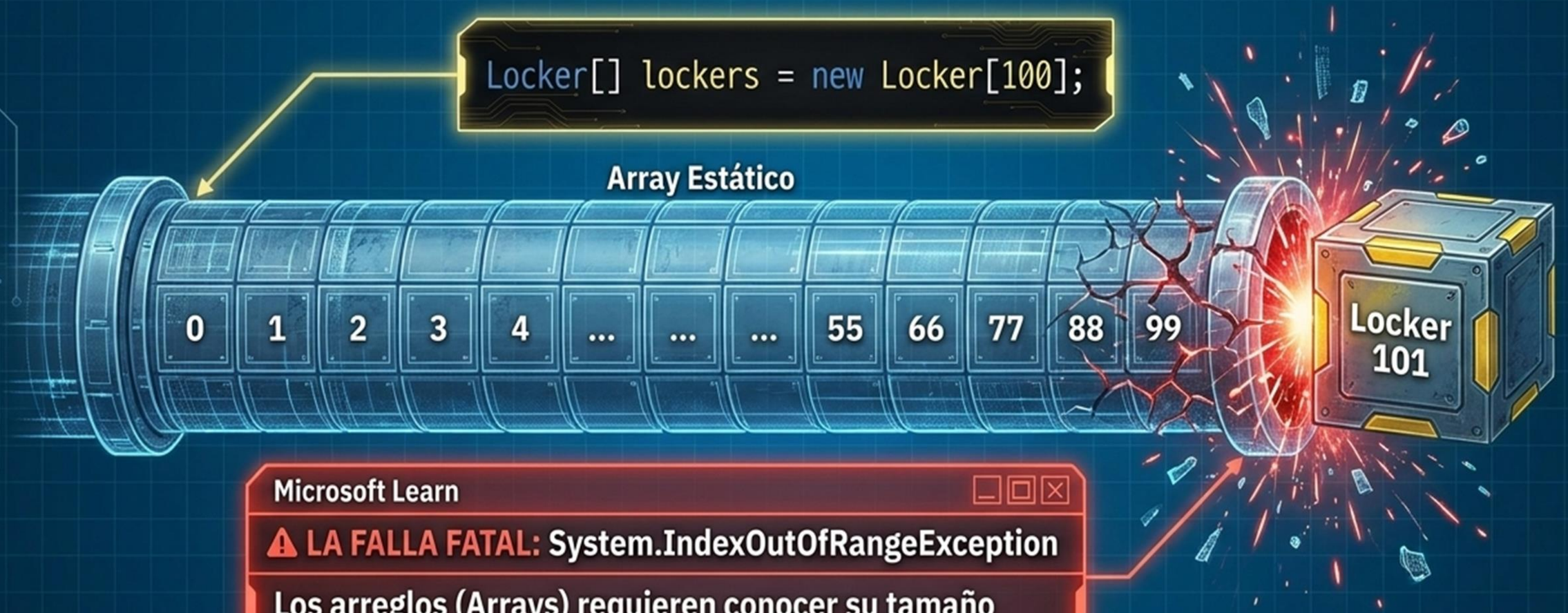
Alias de Tipo
(Atajo Táctico)

```
using Lck = IPVG.Administracion.Infraestructura.Locker;
```

Resolución de ambigüedad y nombres largos en cabecera de archivo

Más Allá de los Límites: La Rigidez del Arreglo

```
Locker[] lockers = new Locker[100];
```



Microsoft Learn

⚠ **LA FALLA FATAL:** System.IndexOutOfRangeException

Los arreglos (Arrays) requieren conocer su tamaño exacto en tiempo de compilación. No pueden crecer. El mundo real no es estático; necesitamos almacenamiento dinámico en el Heap.

El Falso Héroe: ArrayList

La Ventaja:

Crece dinámicamente.

¡No más excepciones de límite!



El Peligro
(Falta de Type Safety)

Un ArrayList acepta CUALQUIER COSA (object).

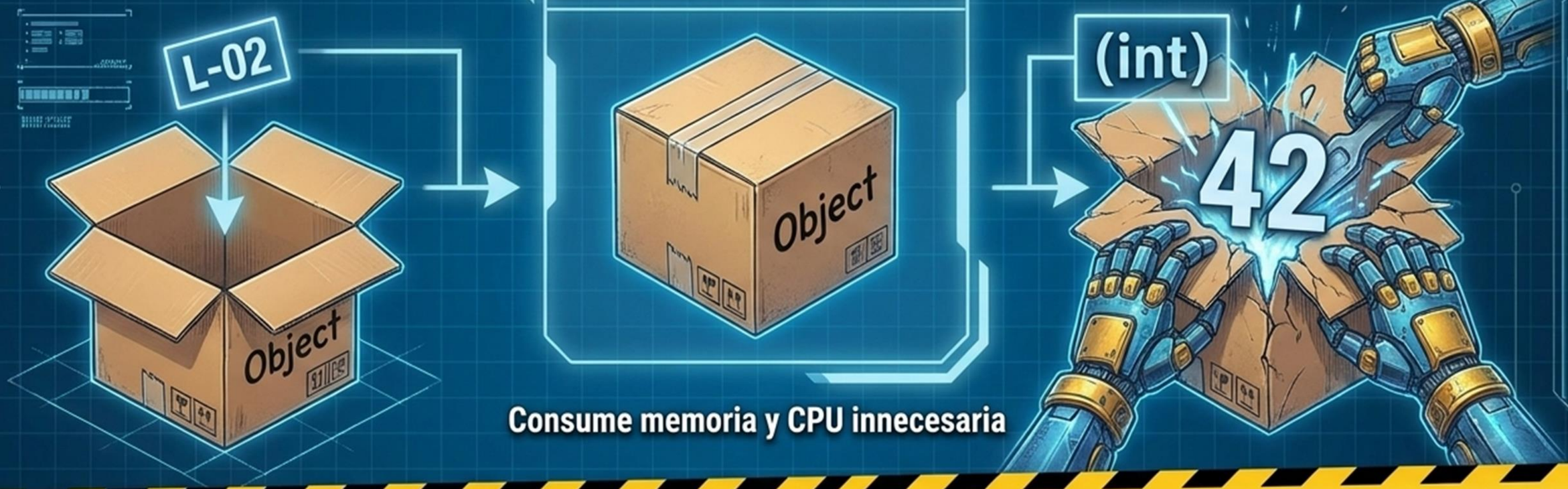
Si agregamos un número y un casillero, el compilador lo permite.

Pero intentar sacar el objeto y equivocarse hará explotar el programa en tiempo de ejecución.

El Costo Oculto: Boxing y Unboxing

Boxing (Empaquetado) -
Convierte Valor a Referencia

Unboxing (Casting) -
Forzar recuperación de tipo



Consume memoria y CPU innecesaria

Conclusión de Ingeniería: Es una operación costosa, ineficiente y propensa a errores (ej. intentar desempacar un candado como si fuera un estudiante).

El Verdadero Héroe: Colecciones Genéricas List<T>

```
List<Locker> listaLockers = new List<Locker>();
```



Type Safety:
La letra <T> es el 'Molde'. Al declarar Locker, el compilador bloquea que ingresen otros datos.

Máxima Velocidad:
Cero cajas de cartón. Cero Boxing/Unboxing. Todo encaja perfecto. Errores bloqueados mientras escribes código.

Escalabilidad:
Expansión infinita y automática en el Heap.

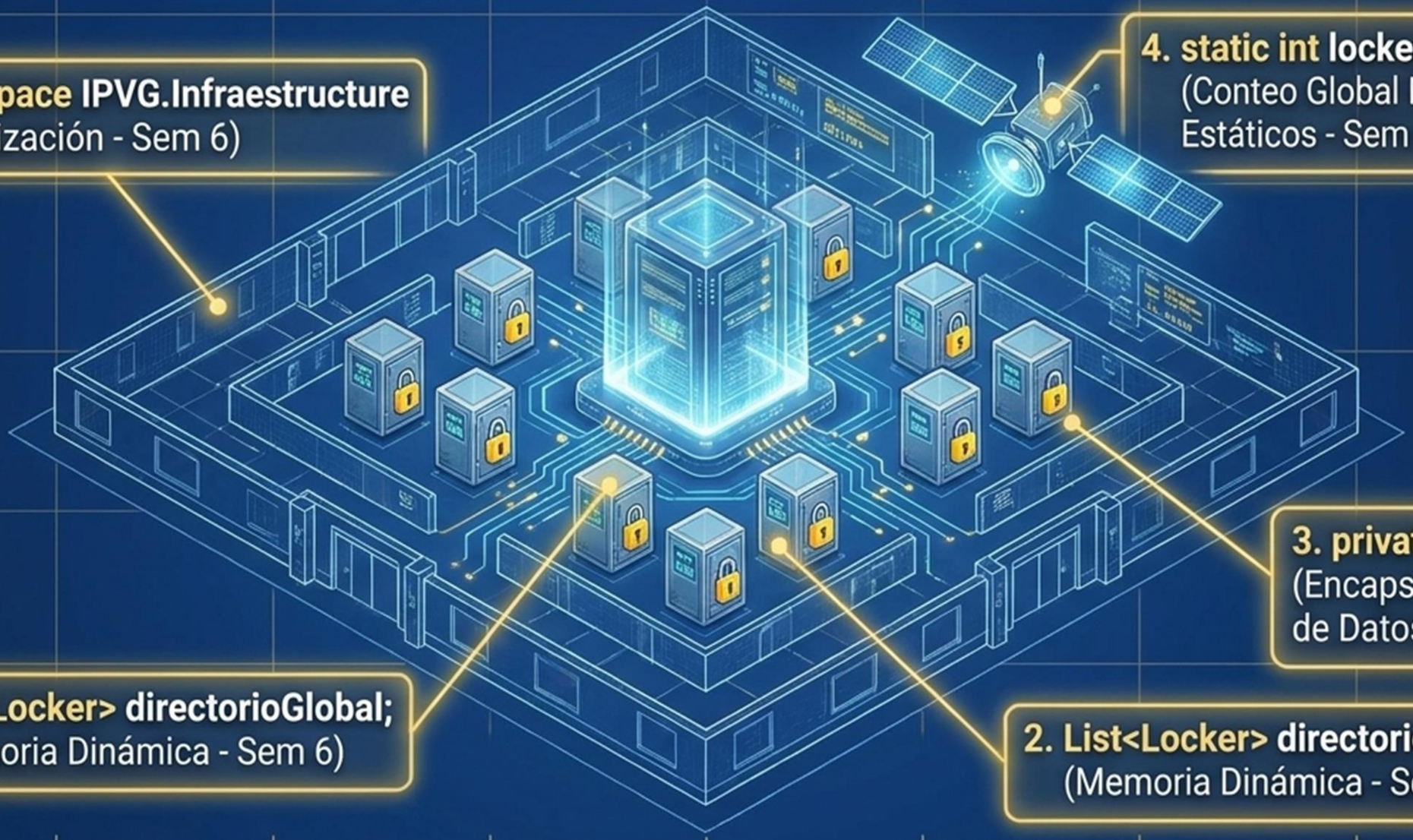
Matriz de Decisión: Estructuras de Datos

Característica	Locker[] (Array)	ArrayList	List<T> (Genérica)
Tamaño	Fijo (Rígido)	Dinámico	Dinámico
Type Safety	Sí (Seguro) ✓	No (Acepta object) ✗	Sí (Seguro) ✓
Boxing/Unboxing	No	Sí (Lento/Costo) ⚠	No (Rápido)
Prevención Errores	Tiempo de Compilación	Tiempo de Ejecución (Peligro)	Tiempo de Compilación

Validado

Veredicto Institucional: En .NET 8, List<T> es el estándar absoluto para el manejo dinámico de memoria.

Arquitectura Final: El Sistema Integrado



1. namespace IPVG.Infraestructure
(Organización - Sem 6)

4. static int lockersActivos;
(Conteo Global Miembros Estáticos - Sem 4)

3. private set
(Encapsulamiento de Datos - Sem 3)

2. List<Locker> directorioGlobal;
(Memoria Dinámica - Sem 6)

2. List<Locker> directorioGlobal;
(Memoria Dinámica - Sem 6)

Desafío Práctico: Refactorización Estructural



Fase 1: Estructura y Estado (Workshop)

```
1  // 1. Declarar el File-scoped Namespace
2  [          ] IPVG.Infraestructura;
3
4  public class Locker {
5      // 2. Miembro estático para rastreo global de todas las instancias
6      private [          ] int contadorTotal = 0;
7
8      // 3. Propiedad encapsulada: modificable sólo de forma interna
9      public string Estado { get; [          ] set; }
10
11     public Locker() {
12         contadorTotal++;
13         Estado = "Disponible";
14     }
15 }
```

¿Qué palabra clave va aquí para zonificar el edificio?

¿Cómo logramos pertenencia global de planta física?

¿Qué modificador protege al setter?

Fase 2: Almacenamiento Dinámico Seguro

```
1  using System;
2  // 1. Habilitar la importación del puente de colecciones tipo seguro
3  using System.Collections.[          ];
4  using IPVG.Infraestructura;
5
6  class Program {
7      static void Main() {
8          // 2. Instanciar la colección genérica correcta para múltiples lockers
9          [    ]<Locker> moduloA = new [    ]<Locker>();
10
11         // 3. Agregar elementos al heap sin Boxing/Unboxing
12         moduloA.[      ](new Locker());
13         moduloA.[      ](new Locker());
14
15         Console.WriteLine($"Lockers operativos en sede: {moduloA.Count}");
16     }
17 }
```

¿De dónde importamos Type Safety?

¿Cuál es el 'Verdadero Héroe' que reemplaza ArrayList?

¿Cuál método adjunta un elemento dinámico?

Fase 3: La Interfaz Interactiva




```
int opcion;
do {
    Console.WriteLine("1. Asignar | 2. Liberar |
        3. Estado | 4. Salir");
    opcion = int.Parse(Console.ReadLine());

    switch (opcion) {
        case 1:
            /* Lógica para moduloA.Add() */
            break;
        case 2:
            // ...
    }
} while (opcion != 4);
```

***(Se invita a todos los alumnos a encender sus IDEs y completar el flujo del Switch en sus estaciones de trabajo basadas en los requerimientos del IPVG)**